Escola Superior de Tecnologia – IPCA
Licenciatura em Engenharia de Sistemas Informáticos
Integração de Sistemas de Informação · 2026/27

Trabalho Prático I

Gestão de Alunos e Notas

Processo ETL com Pentaho Kettle, PostgreSQL e Node-RED

Autor: Bruno Dantas
Número de aluno: 27958
Docente: Óscar Ribeiro

1. Enquadramento

Este relatório documenta o Trabalho Prático I da unidade curricular de Integração de Sistemas de Informação (ISI), no qual o objetivo é aplicar e experimentar ferramentas de ETL (Extract, Transform and Load) na integração de sistemas de informação ao nível dos dados.

Equipa	Bruno Dantas (n.º 27958), trabalho individual
Unidade curricular	Integração de Sistemas de Informação (ISI), 2026/27
Curso	Licenciatura em Engenharia de Sistemas Informáticos
Instituição	Escola Superior de Tecnologia, Instituto Politécnico do Cávado e do Ave (IPCA)
Docentes	Óscar Ribeiro
Tema escolhido	Gestão de alunos e notas
Plataformas	Pentaho Kettle (Spoon) para as transformações e o job; Node-RED para o dashboard; PostgreSQL como base de dados de destino
Repositório Git	https://github.com/BrunoDantas1/ISI26-TP1-GestaoAlunosNotas.git

O processo foi estruturado nas três fases obrigatórias do enunciado: extract (**E**), transofrm (**T**) e load (**L**). Todas as fases são manipuladas por um job do Kettle.

2. Problema

Imaginando que uma instituição de ensino precisa de consolidar informação de alunos e de avaliações proveniente de sistemas distintos: a lista de alunos vem de um serviço externo, as notas chegam em ficheiros exportados manualmente e o plano curricular está descrito num ficheiro XML. Estes dados têm formatos diferentes e problemas de qualidade, pelo que não podem ser usados diretamente para tirar conclusões.

O trabalho pretende demonstrar um processo automático que integra, limpa, valida e cruza estes dados, os carrega numa base de dados relacional, exporta resultados, notifica por email e apresenta indicadores num dashboard. As perguntas a que o processo responde são:

- Quantas avaliações têm estado Aprovado e quantas têm estado Reprovado?
- Qual é a média de cada aluno e de cada unidade curricular (UC)?
- Que alunos e UCs concentram mais reprovações?
- Que registos são inválidos e porquê?

2.1 Fontes de dados

Fonte	Formato	Descrição
API Random User	JSON	Serviço REST que devolve 50 utilizadores fictícios (`seed=isi2026`, para os resultados serem repetíveis). Usada como lista de alunos. Usam-se as nacionalidades `br,es,fr,gb`.
`notas.csv`	CSV	204 registos de notas (username, UC, nota, data, época), gerados com erros introduzidos de propósito.
`ucs.xml`	XML	Seis unidades curriculares com código, nome, ECTS e ano.

2.2 Problemas de qualidade tratados

Problema	Exemplo
Texto sem normalização	Usernames em maiúsculas ou com espaços; códigos de UC como `uc02` ou `UC03`
Formato numérico inconsistente	Notas com vírgula decimal (`14,5`)
Valores inválidos ou em falta	Notas fora do intervalo 0–20 (`27`, `35`, `-3`) e notas vazias
Formatos de data diferentes	`15/02/2026` em vez de `2026-02-15`
Registos duplicados	3 linhas repetidas no ficheiro de notas
Integridade referencial	Nota de um username inexistente nos alunos (`fantasmaaluno99`)
Dados pessoais	Emails que devem ser anonimizados

3. Estratégia utilizada

3.1 Arquitetura do processo

Fase	O que faz	Formatos	Ferramenta
E	Lê alunos (API REST), notas (ficheiro) e UCs (ficheiro)	JSON, CSV, XML	Kettle: REST client, JSON input, CSV file input, Get data from XML
T	Limpeza, regex, validação, junção, agregação, cálculo do estado	Interno (linhas Kettle)	Kettle: String operations, Replace in string, Filter rows, Stream lookup, Group by, entre outros
L	Base de dados, ficheiros de exportação e erros, email e dashboard	PostgreSQL, CSV, email, web	Kettle: Table output, Text file output, Mail; Node-RED

Para que cada fase fique independente e possa ser testada isoladamente, o processo foi dividido em **três transformações** encadeadas por um **job**. Cada transformação lê os ficheiros produzidos pela anterior:

```
t1_alunos_api   : API (JSON)  ->  limpeza  ->  alunos_limpos.csv
t2_notas_ucs    : notas.csv   ->  limpeza  ->  notas_limpas.csv
t3_resultados   : alunos + notas + UCs (XML)  ->  PostgreSQL + CSV
j_principal     : t1 -> t2 -> t3 -> email   (com ramo de erro)
```

3.2 Ferramentas

Ferramenta	Utilização
Pentaho Data Integration 10.2 (Kettle/Spoon), Java 11	Desenho e execução das transformações (`.ktr`) e do job (`.kjb`)
PostgreSQL 18 e pgAdmin 4	Base de dados de destino `isi_notas` e verificação dos dados carregados
Driver JDBC `postgresql-42.7.14`	Ligação do Kettle ao PostgreSQL
Node-RED com `node-red-dashboard` e `node-red-contrib-postgresql`	Dashboard com gráficos alimentados diretamente pela base de dados
Servidor SMTP do Gmail	Envio do email com a lista de reprovados em anexo

3.3 Expressões regulares

As expressões regulares são usadas em quatro pontos do processo:

Objetivo	Expressão	Efeito
Validar o email	`^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$`	Passo Regex Evaluation; emails inválidos seguem para o ficheiro de erros
Extrair o ano de nascimento	Procurar `^(\d{4}).*$`, substituir por `$1`	`1948-10-20T17:25:42Z` passa a `1948`
Anonimizar o email	Procurar `^(.{3}).*@(.*)$`, substituir por `$1***@$2`	`maxence.leclerc@example.com` passa a `max***@example.com`
Normalizar a data	Procurar `^(\d{2})/(\d{2})/(\d{4})$`, substituir por `$3-$2-$1`	`15/02/2026` passa a `2026-02-15`

3.4 Cumprimento dos critérios do enunciado

Critério	Onde está no trabalho
Diversidade de formatos (2 a 3)	JSON (API), CSV (notas) e XML (UCs); exportação em CSV e SQL (PostgreSQL)
Diversidade de fontes	API REST, ficheiros locais e base de dados relacional
Expressões regulares	Validação, extração, anonimização e normalização (secção 3.3)
Joins, lookups e agrupamentos	Dois Stream lookup (UCs e alunos) e Group by (média por aluno)
Operações sobre valores	Trim, maiúsculas/minúsculas, concatenação, conversão de tipos, cálculo, ordenação, filtragem, remoção de duplicados, anonimização
Job com acesso a serviços remotos	Job que orquestra três transformações e envia email via SMTP
Geração de logs	Ficheiros de erros (`erros_notas`, `erros_alunos`, `erros_orfaos`) e passo Write to log no ramo de erro do job
Acesso a APIs	Random User API (REST/JSON)
Operações sobre bases de dados	Criação das tabelas, carregamento com Table output (Truncate) e consultas
Dashboard	Node-RED Dashboard com gráficos lidos do PostgreSQL

4. Transformações

4.1 t1_alunos_api: alunos a partir da API

Objetivo: obter os alunos através de uma chamada REST, normalizar os dados, validar o email, anonimizá-lo e guardar o resultado em `alunos_limpos.csv`.

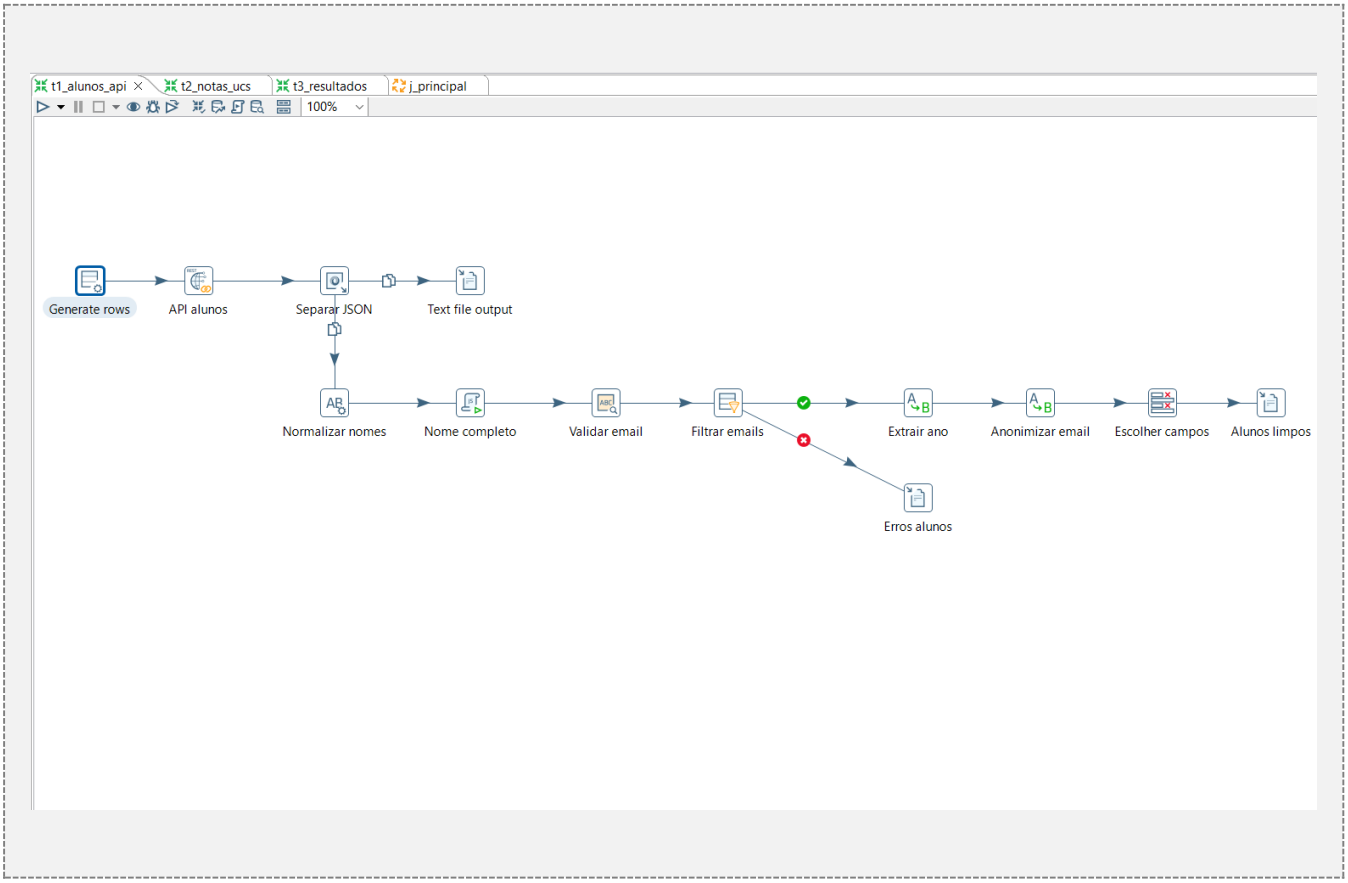
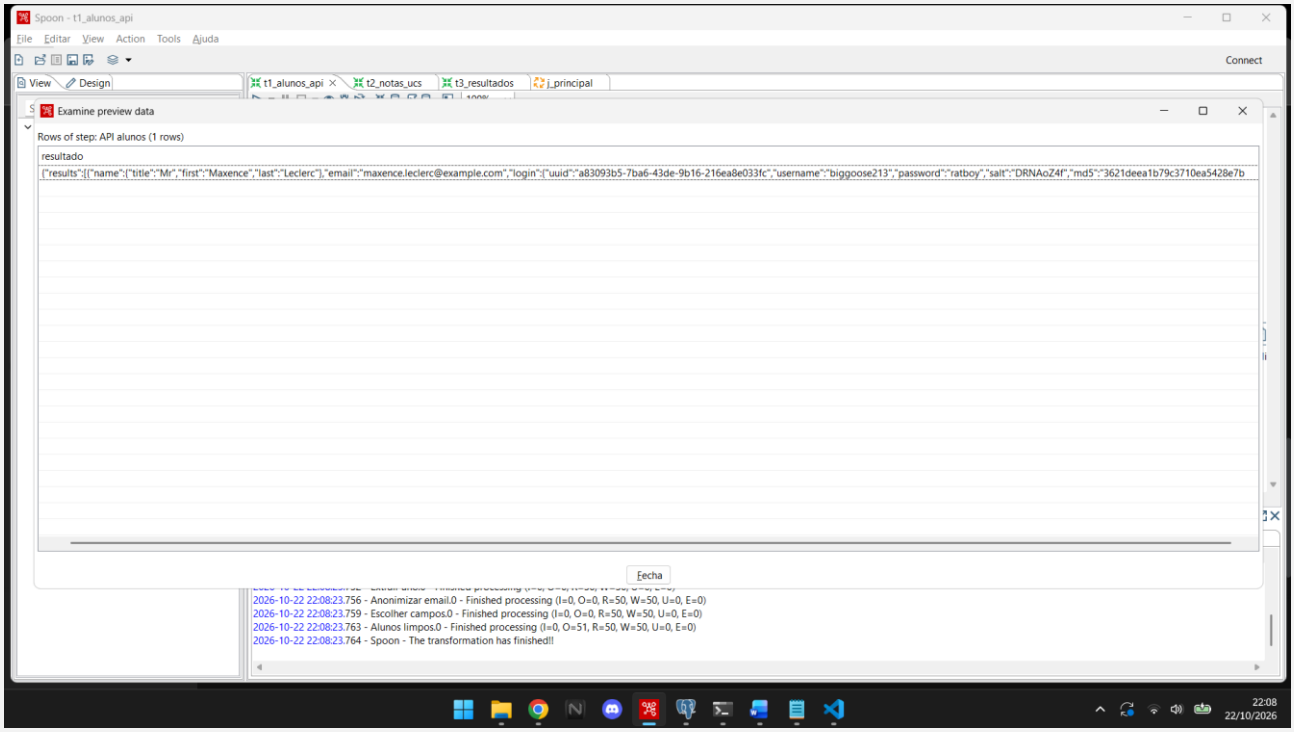


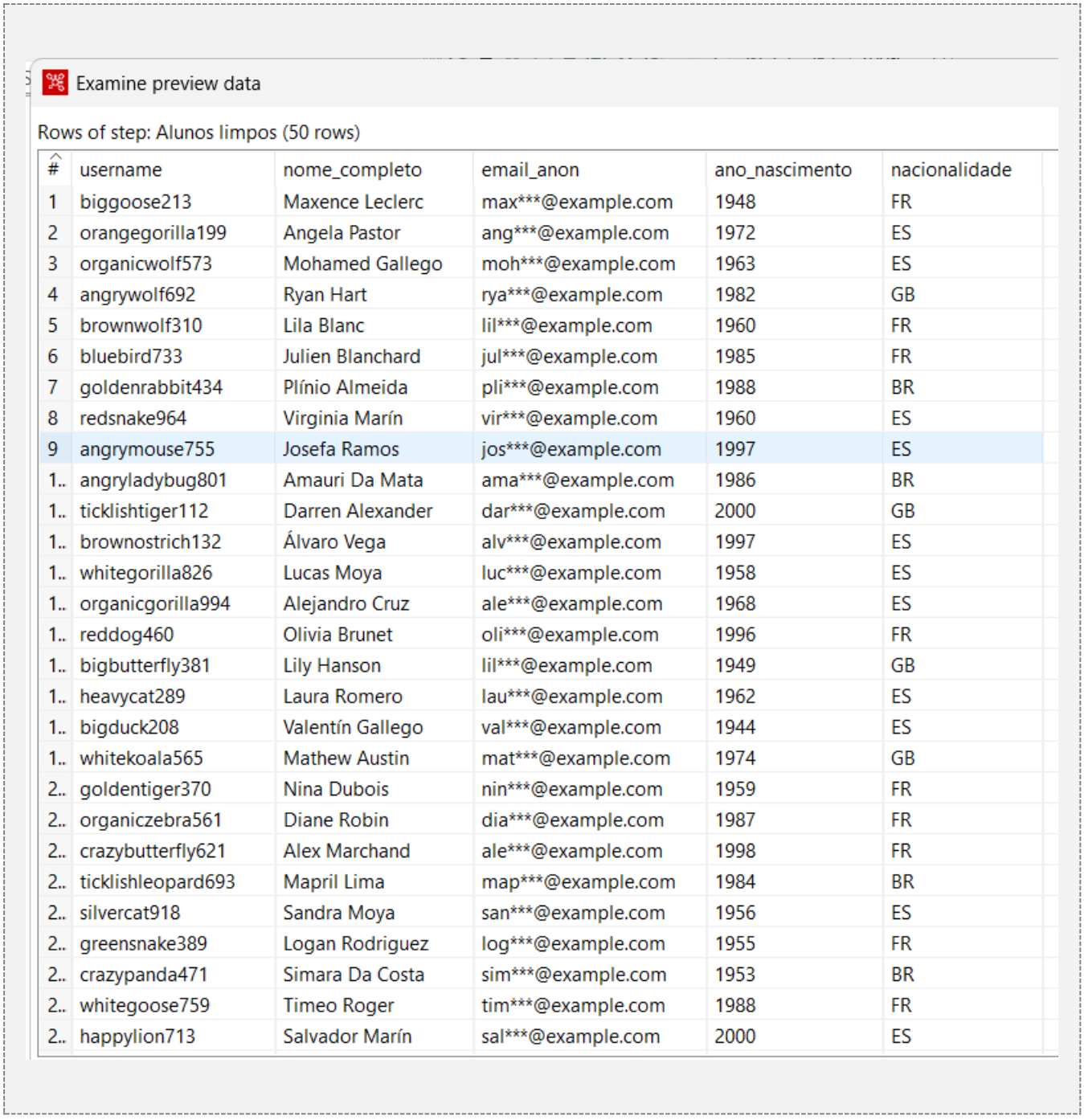
Figura 1 – Diagrama da transformação t1_alunos_api no Spoon

Passo	Tipo	Função
Generate rows	Generate rows	Gera uma linha para disparar o REST client, que exige uma linha de entrada
API alunos	REST client	GET à Random User API; o JSON fica no campo `resultado`
Separar JSON	JSON input	Extrai 6 campos com JSONPath (`$.results[*].name.first`, `email`, `dob.date`, `nat`, `login.username`, …). Envia as linhas em modo Copy para dois destinos
Text file output	Text file output	Exporta os dados em bruto para `alunos_api.csv`
Normalizar nomes	String operations	Remove espaços; primeira letra maiúscula nos nomes; email em minúsculas; nacionalidade em maiúsculas
Nome completo	Modified JavaScript Value	Calcula `nome_completo = primeiro_nome + " " + apelido`
Validar email	Regex Evaluation	Aplica a expressão regular de validação e cria `email_valido`
Filtrar emails	Filter rows	Emails válidos seguem; inválidos vão para `Erros alunos`
Erros alunos	Text file output	Grava `erros_alunos.csv`
Extrair ano	Replace in string	Regex que extrai o ano de nascimento para `ano_nascimento`

Passo	Tipo	Função
Anonimizar email	Replace in string	Regex que oculta a parte local do email em `email_anon`
Escolher campos	Select values	Mantém `username`, `nome_completo`, `email_anon`, `ano_nascimento`, `nacionalidade`
Alunos limpos	Text file output	Grava `alunos_limpos.csv`

Resultado: 50 alunos lidos e 50 gravados. O ramo de erros ficou vazio porque a API só devolve emails válidos, mas foi mantido para demonstrar a validação.





#	username	nome_completo	email_anon	ano_nascimento	nacionalidade
1	biggoose213	Maxence Leclerc	max***@example.com	1948	FR
2	orangegorilla199	Angela Pastor	ang***@example.com	1972	ES
3	organicwolf573	Mohamed Gallego	moh***@example.com	1963	ES
4	angrywolf692	Ryan Hart	rya***@example.com	1982	GB
5	brownwolf310	Lila Blanc	lil***@example.com	1960	FR
6	bluebird733	Julien Blanchard	jul***@example.com	1985	FR
7	goldenrabbit434	Plínio Almeida	pli***@example.com	1988	BR
8	redsnake964	Virginia Marín	vir***@example.com	1960	ES
9	angrymouse755	Josefa Ramos	jos***@example.com	1997	ES
1..	angryladybug801	Amauri Da Mata	ama***@example.com	1986	BR
1..	ticklishtiger112	Darren Alexander	dar***@example.com	2000	GB
1..	brownostrich132	Álvaro Vega	alv***@example.com	1997	ES
1..	whitegorilla826	Lucas Moya	luc***@example.com	1958	ES
1..	organicgorilla994	Alejandro Cruz	ale***@example.com	1968	ES
1..	reddog460	Olivia Brunet	oli***@example.com	1996	FR
1..	bigbutterfly381	Lily Hanson	lil***@example.com	1949	GB
1..	heavycat289	Laura Romero	lau***@example.com	1962	ES
1..	bigduck208	Valentín Gallego	val***@example.com	1944	ES
1..	whitekoala565	Mathew Austin	mat***@example.com	1974	GB
2..	goldentiger370	Nina Dubois	nin***@example.com	1959	FR
2..	organiczebra561	Diane Robin	dia***@example.com	1987	FR
2..	crazybutterfly621	Alex Marchand	ale***@example.com	1998	FR
2..	ticklishleopard693	Mapril Lima	map***@example.com	1984	BR
2..	silvercat918	Sandra Moya	san***@example.com	1956	ES
2..	greensnake389	Logan Rodriguez	log***@example.com	1955	FR
2..	crazypanda471	Simara Da Costa	sim***@example.com	1953	BR
2..	whitegoose759	Timeo Roger	tim***@example.com	1988	FR
2..	happylion713	Salvador Marín	sal***@example.com	2000	ES

Figura 2 – Pré-visualização do resultado da API (JSON) e do ficheiro alunos_limpos.csv

4.2 t2_notas_ucs: limpeza e validação das notas

Objetivo: ler `notas.csv`, corrigir os erros de formato, validar o intervalo das notas, remover duplicados e separar os registos inválidos.

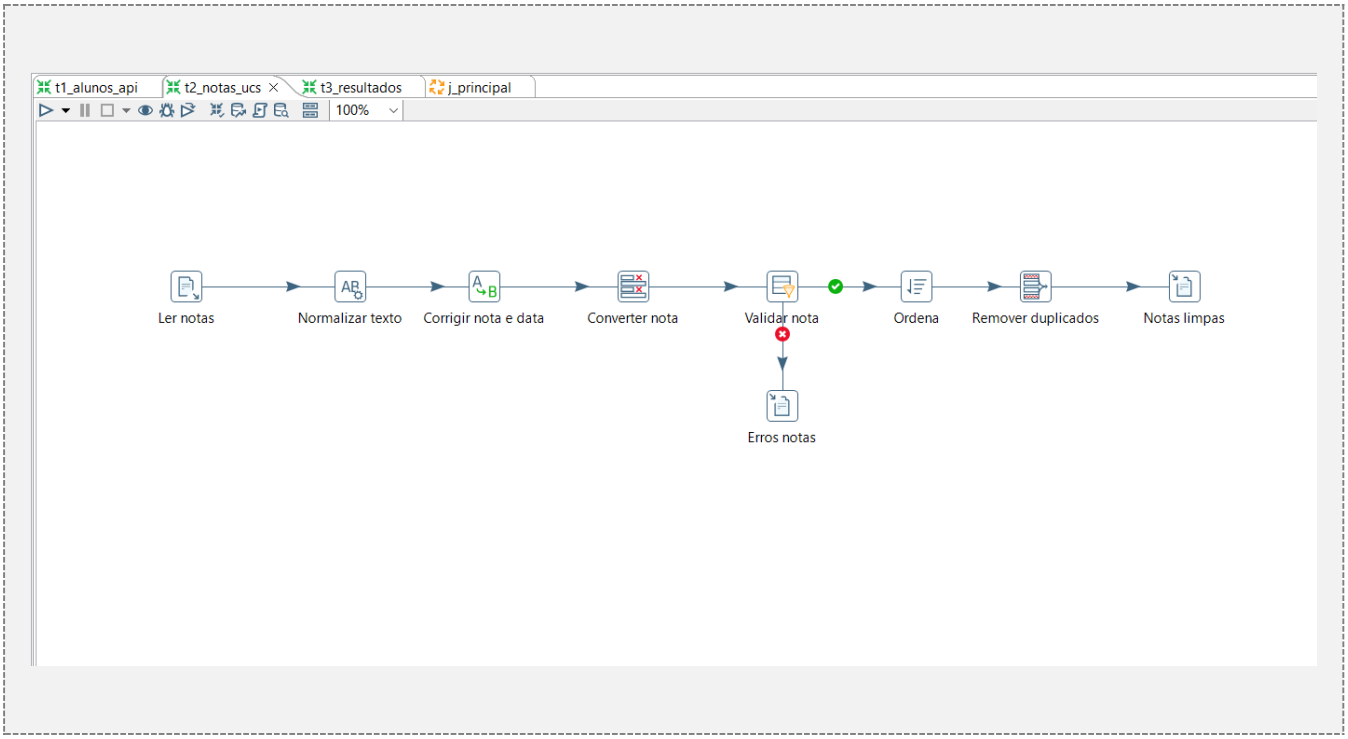
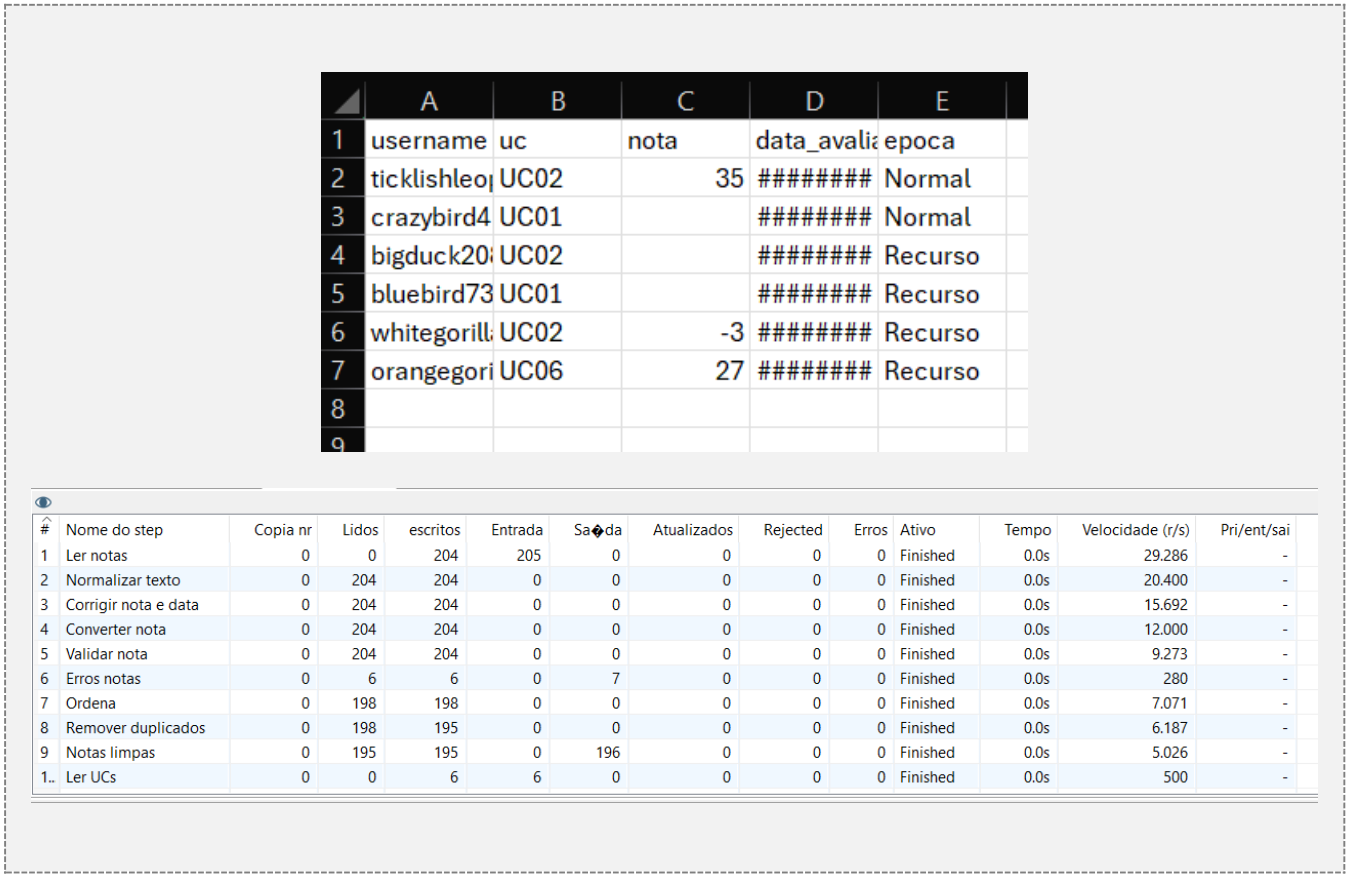


Figura 3 – Diagrama da transformação t2_notas_ucs no Spoon

Passo	Tipo	Função
Ler notas	CSV file input	Lê 204 registos; todos os campos são lidos como texto para evitar falhas de conversão antes da limpeza
Normalizar texto	String operations	`username` sem espaços e em minúsculas; `uc` sem espaços e em maiúsculas
Corrigir nota e data	Replace in string	Troca a vírgula decimal por ponto; regex que converte `dd/mm/aaaa` em `aaaa-mm-dd`
Converter nota	Select values (Meta-data)	Converte `nota` para número (formato `#.#`, decimal `.`)
Validar nota	Filter rows	Aceita notas não nulas entre 0 e 20; as restantes seguem para `Erros notas`
Erros notas	Text file output	Grava `erros_notas.csv` (log de registos inválidos)
Ordena	Sort rows	Ordena pelos cinco campos (requisito do Unique rows)
Remover duplicados	Unique rows	Elimina linhas repetidas
Notas limpas	Text file output	Grava `notas_limpas.csv`

Etapa	Linhas
Registos lidos	204
Rejeitados (3 fora do intervalo, 3 vazios)	6, em `erros_notas.csv`
Válidos antes de remover duplicados	198
Duplicados removidos	3

Etapa	Linhas
Notas limpas	195



	A	B	C	D	E
1	username	uc	nota	data_avali	epoca
2	ticklishleo	UC02	35	########	Normal
3	crazybird4	UC01		########	Normal
4	bigduck20	UC02		########	Recurso
5	bluebird73	UC01		########	Recurso
6	whitegorill	UC02	-3	########	Recurso
7	orangegori	UC06	27	########	Recurso
8					
9					

#	Nome do step	Copia nr	Lidos	escritos	Entrada	Sa&#65533;da	Atualizados	Rejected	Erros	Ativo	Tempo	Velocidade (r/s)	Pri/ent/sai
1	Ler notas	0	0	204	205	0	0	0	0	Finished	0.0s	29.286	-
2	Normalizar texto	0	204	204	0	0	0	0	0	Finished	0.0s	20.400	-
3	Corrigir nota e data	0	204	204	0	0	0	0	0	Finished	0.0s	15.692	-
4	Converter nota	0	204	204	0	0	0	0	0	Finished	0.0s	12.000	-
5	Validar nota	0	204	204	0	0	0	0	0	Finished	0.0s	9.273	-
6	Erros notas	0	6	6	0	7	0	0	0	Finished	0.0s	280	-
7	Ordena	0	198	198	0	0	0	0	0	Finished	0.0s	7.071	-
8	Remover duplicados	0	198	195	0	0	0	0	0	Finished	0.0s	6.187	-
9	Notas limpas	0	195	195	0	196	0	0	0	Finished	0.0s	5.026	-
1..	Ler UCs	0	0	6	6	0	0	0	0	Finished	0.0s	500	-

Figura 4 – Ficheiro erros_notas.csv e Step Metrics da transformação t2

4.3 t3_resultados: cruzamento, cálculos e carregamento

Objetivo: juntar notas, UCs (XML) e alunos, calcular o estado de cada avaliação e a média por aluno, carregar o resultado no PostgreSQL e gerar a lista de reprovados.

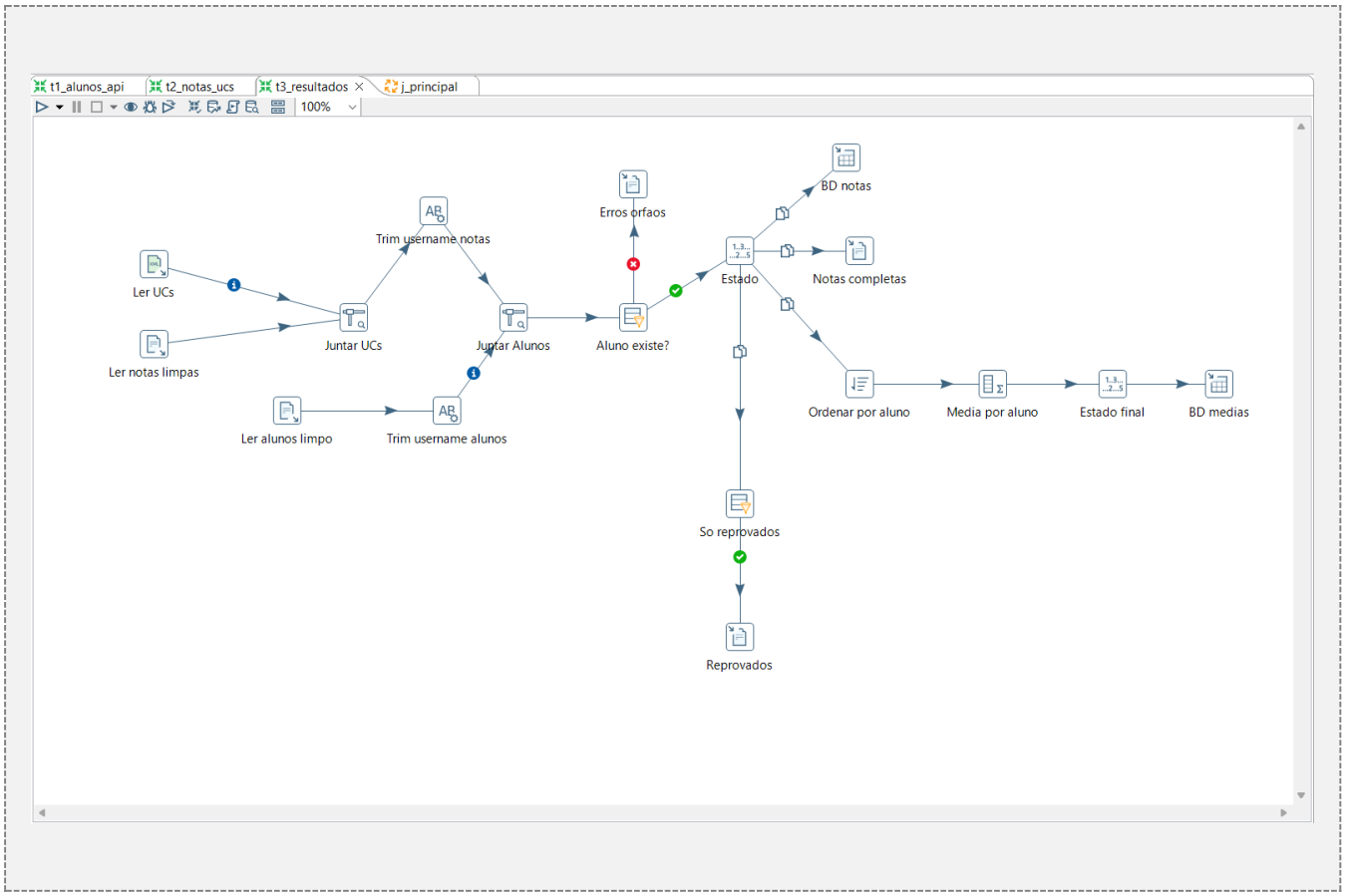
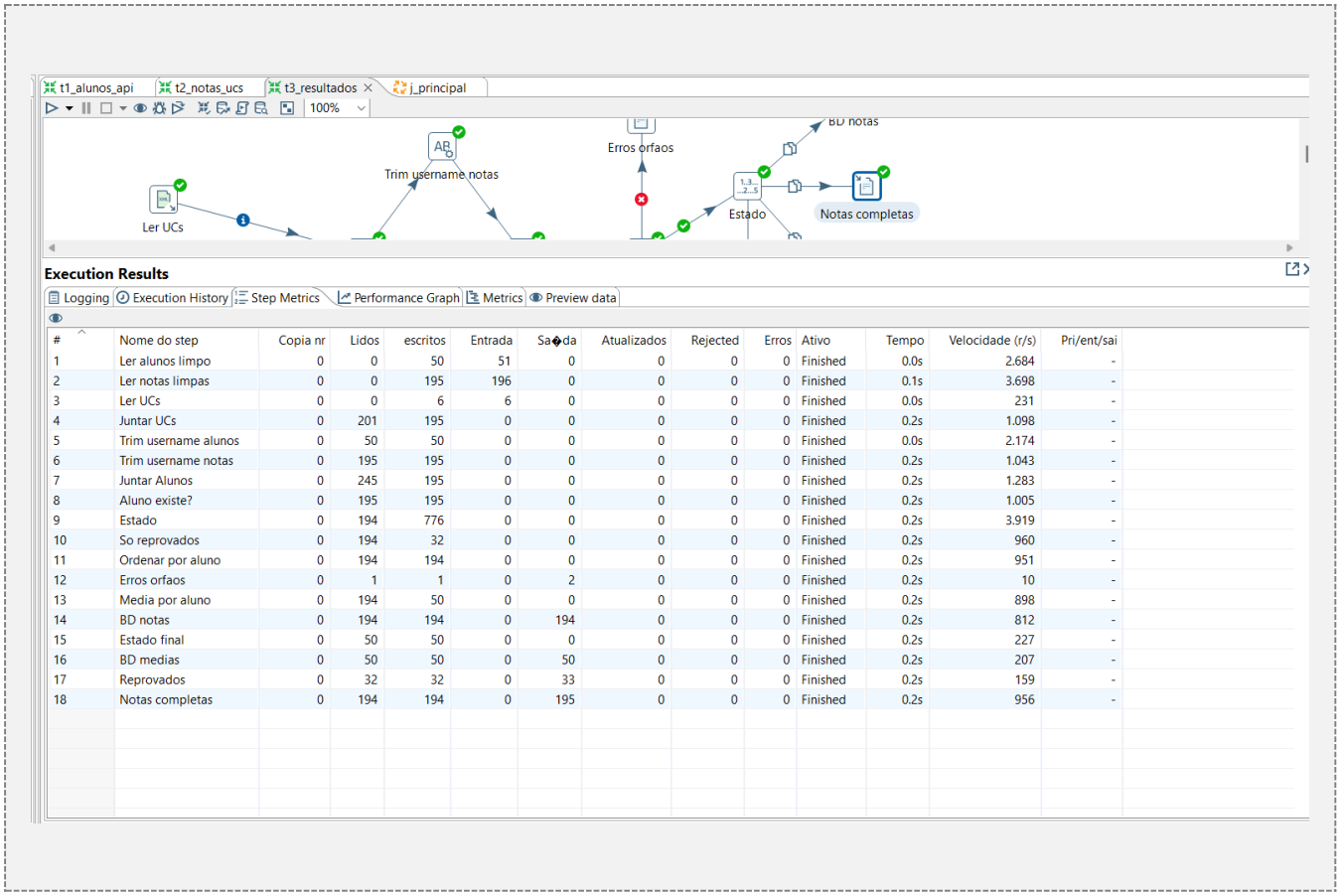


Figura 5 – Diagrama da transformação t3_resultados no Spoon

Passo	Tipo	Função
Ler notas limpas	CSV file input	Lê `notas_limpas.csv` (195 linhas)
Ler UCs	Get data from XML	Lê `ucs.xml` com XPath `/ucs/uc` (6 linhas)
Juntar UCs	Stream lookup	Chave `uc` = `codigo`; traz `uc_nome`, `ects`, `ano_uc`
Trim username notas / alunos	String operations	Garante chaves iguais (minúsculas, sem espaços) antes do join
Ler alunos limpos	CSV file input	Lê `alunos_limpos.csv` (50 linhas)
Juntar Alunos	Stream lookup	Chave `username`; traz `nome_completo`, `email_anon`, `nacionalidade`
Aluno existe?	Filter rows	`nome_completo IS NOT NULL`; os restantes vão para `Erros orfaos`
Erros orfaos	Text file output	Grava `erros_orfaos.csv` (notas sem aluno)
Estado	Number range	Nota de 10 a 20 = `Aprovado`; valor por omissão `Reprovado`
Notas completas	Text file output	Grava `notas_completas.csv`
BD notas	Table output	Carrega `notas_completas` no PostgreSQL (com Truncate table)

Passo	Tipo	Função
So reprovados / Reprovados	Filter rows / Text file output	Filtra `estado = Reprovado` e grava `reprovados.csv`
Ordenar por aluno	Sort rows	Ordena por `username` (necessário para o Group by)
Media por aluno	Group by	Agrupa por aluno: `media` (média), `n_ucs` (contagem), `nota_max` (máximo)
Estado final	Number range	Média de 10 a 20 = `Aprovado`; senão `Reprovado`
BD medias	Table output	Carrega `medias_alunos` no PostgreSQL

Resultado	Valor
Notas lidas	195
Notas sem aluno correspondente (`fantasmaaluno99`)	1, em `erros_orfaos.csv`
Notas completas carregadas em `notas_completas`	194
Estado Aprovado / Reprovado	162 / 32
Alunos em `medias_alunos`	50



#	Nome do step	Copia nr	Lidos	escritos	Entrada	Sa◆da	Atualizados	Rejected	Erros	Ativo	Tempo	Velocidade (r/s)	Pri/ent/sai
1	Ler alunos limpo	0	0	50	51	0	0	0	0	Finished	0.0s	2.684	-
2	Ler notas limpas	0	0	195	196	0	0	0	0	Finished	0.1s	3.698	-
3	Ler UCs	0	0	6	6	0	0	0	0	Finished	0.0s	231	-
4	Juntar UCs	0	201	195	0	0	0	0	0	Finished	0.2s	1.098	-
5	Trim username alunos	0	50	50	0	0	0	0	0	Finished	0.0s	2.174	-
6	Trim username notas	0	195	195	0	0	0	0	0	Finished	0.2s	1.043	-
7	Juntar Alunos	0	245	195	0	0	0	0	0	Finished	0.2s	1.283	-
8	Aluno existe?	0	195	195	0	0	0	0	0	Finished	0.2s	1.005	-
9	Estado	0	194	776	0	0	0	0	0	Finished	0.2s	3.919	-
10	So reprovados	0	194	32	0	0	0	0	0	Finished	0.2s	960	-
11	Ordenar por aluno	0	194	194	0	0	0	0	0	Finished	0.2s	951	-
12	Erros orfaos	0	1	1	0	2	0	0	0	Finished	0.2s	10	-
13	Media por aluno	0	194	50	0	0	0	0	0	Finished	0.2s	898	-
14	BD notas	0	194	194	0	194	0	0	0	Finished	0.2s	812	-
15	Estado final	0	50	50	0	0	0	0	0	Finished	0.2s	227	-
16	BD medias	0	50	50	0	50	0	0	0	Finished	0.2s	207	-
17	Reprovados	0	32	32	0	33	0	0	0	Finished	0.2s	159	-
18	Notas completas	0	194	194	0	195	0	0	0	Finished	0.2s	956	-

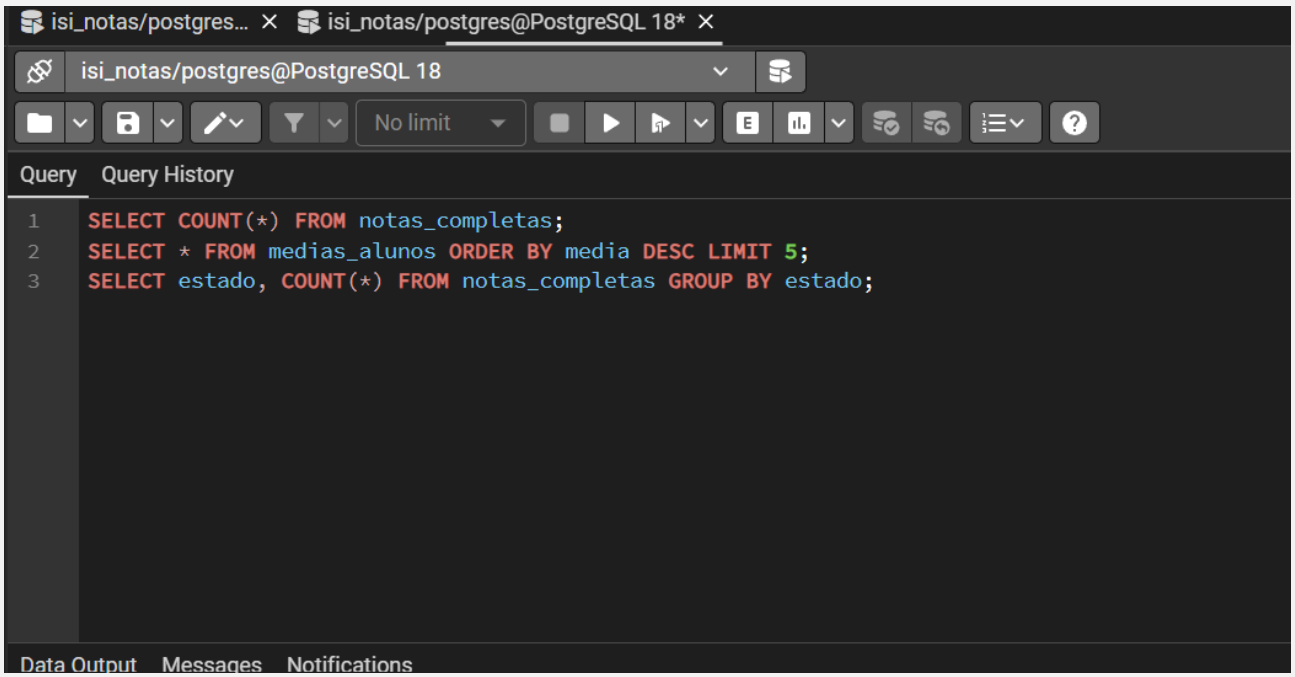


Figura 6 – Step Metrics da t3 e consulta no pgAdmin (SELECT estado, COUNT(*) ...)

4.4 Base de dados de destino

A base de dados `isi_notas` (PostgreSQL) contém duas tabelas, criadas com o script `src/criar_tabelas.sql`:

```
CREATE TABLE notas_completas (
  username VARCHAR(50), uc VARCHAR(10), nota NUMERIC(4,1),
  data_avaliacao VARCHAR(10), epoca VARCHAR(20),
  uc_nome VARCHAR(100), ects INTEGER, ano_uc INTEGER,
  nome_completo VARCHAR(100), email_anon VARCHAR(100),
  nacionalidade VARCHAR(5), estado VARCHAR(20)
);

CREATE TABLE medias_alunos (
  username VARCHAR(50), nome_completo VARCHAR(100),
  media NUMERIC(5,2), n_ucs INTEGER, nota_max NUMERIC(4,1),
  estado_final VARCHAR(20)
);
```

Os passos Table output usam a opção Truncate table, por isso cada execução do job substitui os dados anteriores em vez de os duplicar.

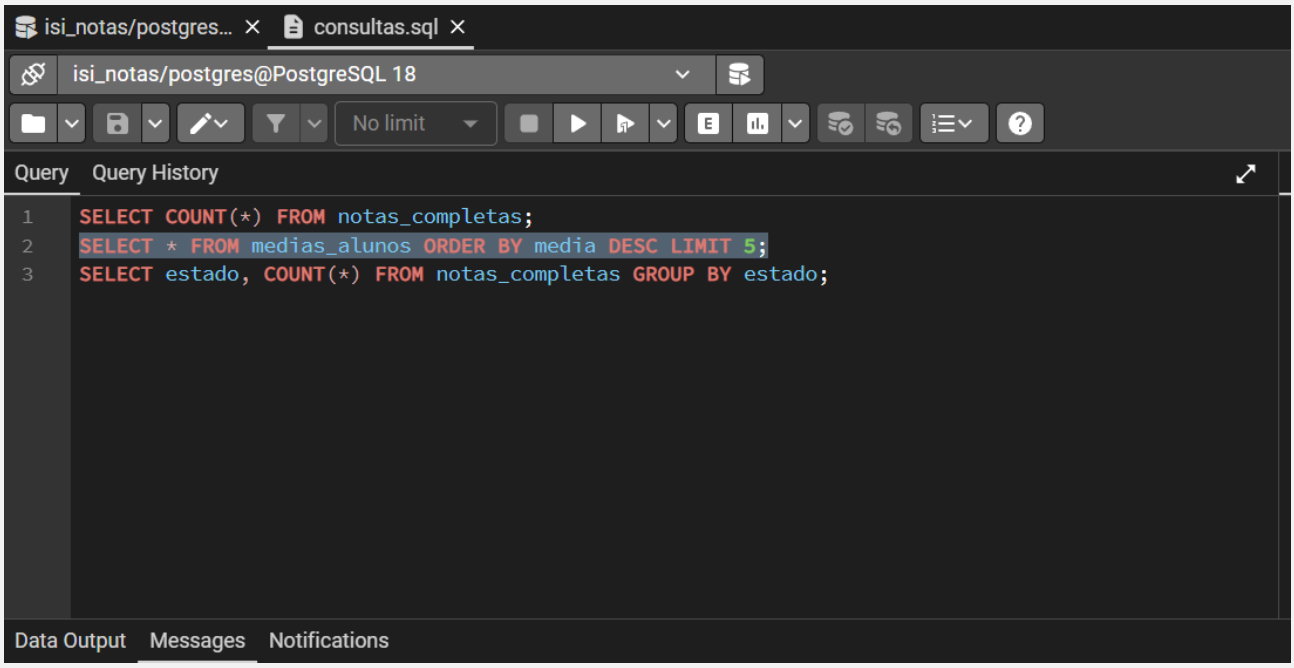


*Figura 7 – pgAdmin: tabela medias_alunos (SELECT * FROM medias_alunos ORDER BY media DESC)*

5. Jobs

5.1 Job principal (j_principal)

O job `j_principal` orquestra o processo completo. As três transformações executam-se em sequência e cada uma só arranca se a anterior terminar com sucesso:

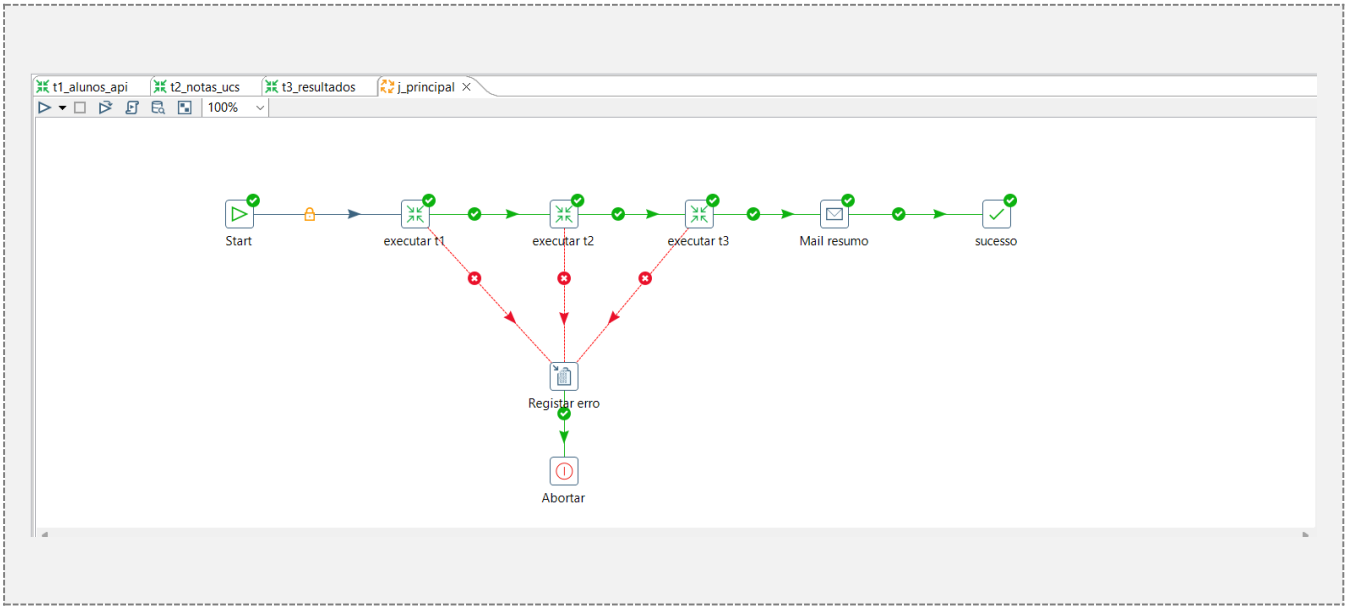


Figura 8 – Diagrama do job j_principal no Spoon

Entrada do job	Tipo	Função
Start	Start	Início da execução
executar t1 / t2 / t3	Transformation	Executam os ficheiros `.ktr` pela ordem correta
Mail resumo	Mail	Envia o email com a lista de reprovados
sucesso	Success	Termina o job com sucesso
Registar erro	Write to log	Regista a falha no log (nível Error)
Abortar	Abort job	Termina o job com erro

5.2 Tratamento de erros e logs

Cada uma das três transformações tem uma ligação a vermelho (follow when result is false) para o passo Registar erro, seguido de Abortar. Assim, uma falha numa transformação deixa registo no log e impede que as fases seguintes corram com dados incompletos. Para além deste log do job, o processo gera três ficheiros de erros ao nível dos dados:

- `erros_notas.csv`: notas vazias ou fora do intervalo 0–20;
- `erros_alunos.csv`: alunos com email inválido;
- `erros_orfaos.csv`: notas cujo username não existe na lista de alunos.

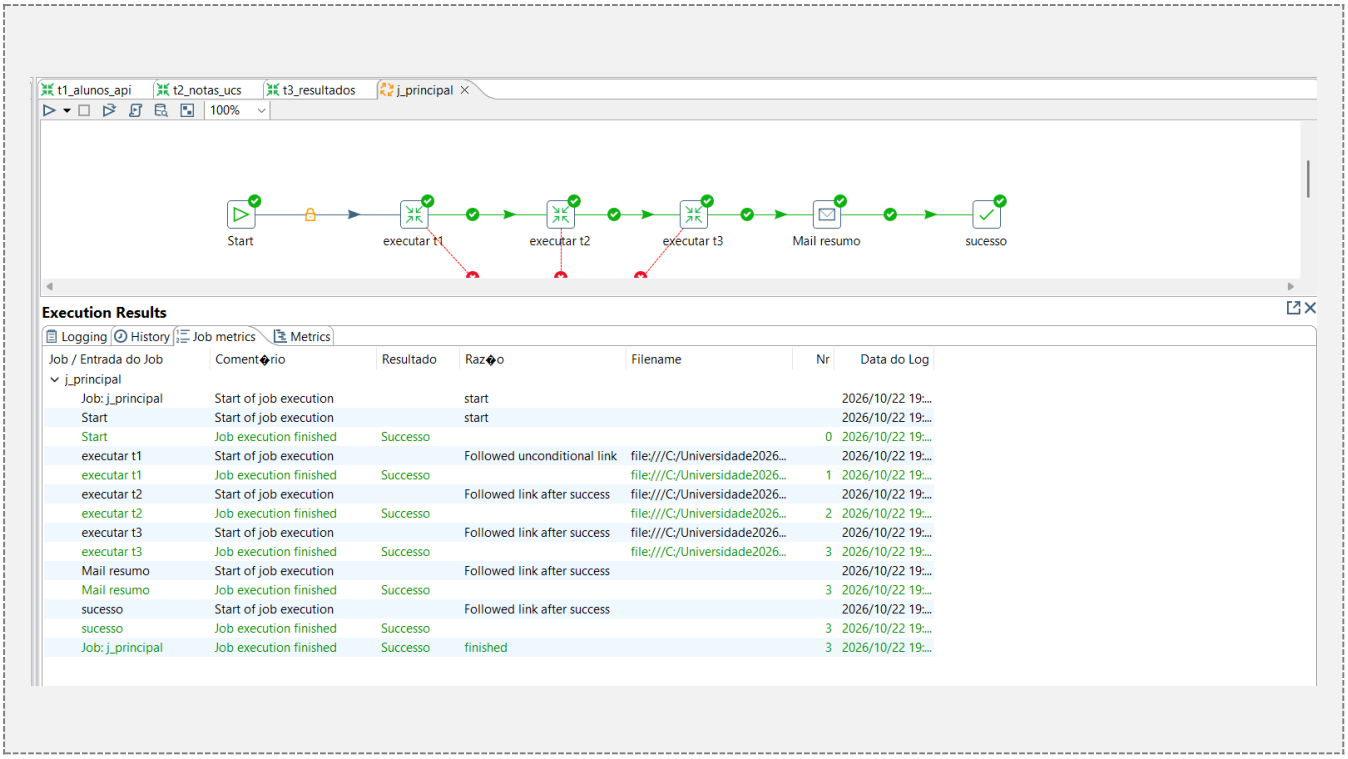


Figura 9 – Job metrics com todas as entradas a verde (resultado: Sucesso)

5.3 Notificação por email

No fim da `t3`, o passo **Mail** envia, através do servidor SMTP do Gmail (porta 587, TLS, palavra-passe de aplicação), um email com o ficheiro `reprovados.csv` em anexo. O ficheiro lista as 32 avaliações com estado Reprovado e serve de alerta para o acompanhamento dos alunos.

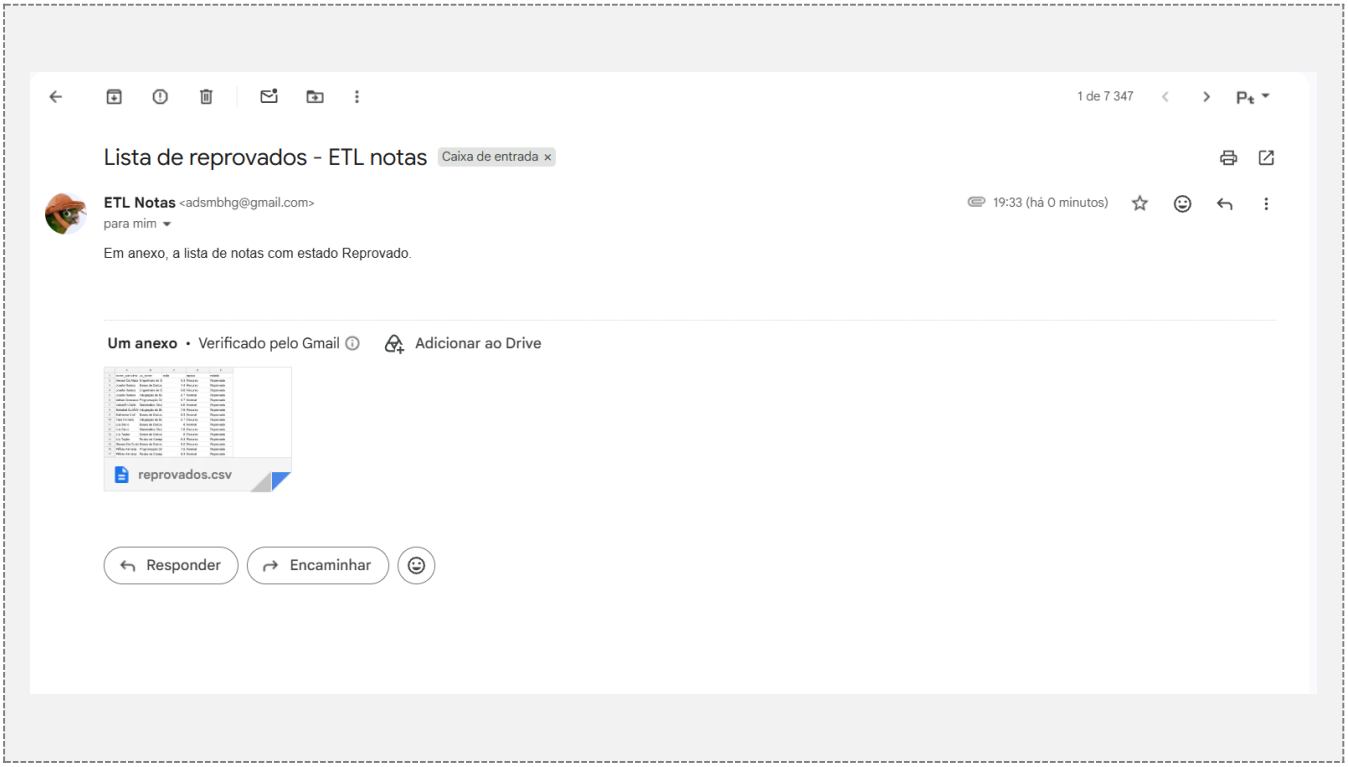


Figura 10 – Email recebido com o anexo reprovados.csv

5.4 Dashboard em Node-RED

O Node-RED liga-se diretamente à base de dados `isi_notas` através do nó `postgresql`. Cada gráfico segue o mesmo padrão: um inject que repete a cada 30 segundos, uma consulta SQL, uma função que transforma as linhas em pontos do gráfico e um chart. Os gráficos criados são:

Gráfico	Tipo	Consulta
Aprovados vs Reprovados	Circular	`SELECT estado, COUNT(*)::int AS total FROM notas_completas GROUP BY estado;`
Média por UC	Barras horizontais	`SELECT uc_nome, ROUND(AVG(nota),2)::float AS media FROM notas_completas GROUP BY uc_nome ORDER BY uc_nome;`
Reprovação por UC	Barras horizontais	`SELECT uc_nome, COUNT(*)::int AS total FROM notas_completas WHERE estado='Reprovado' GROUP BY uc_nome ORDER BY uc_nome;`

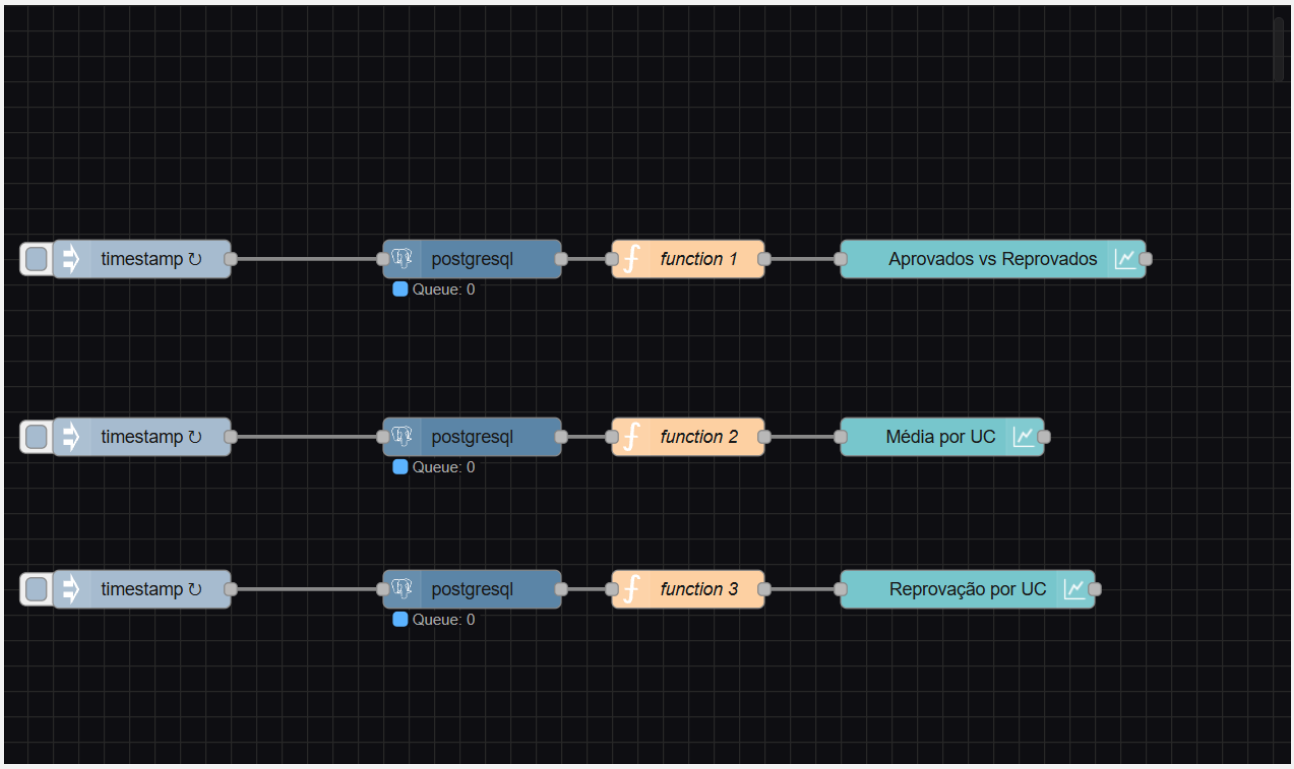


Figura 11 – Flow do Node-RED (inject, postgresql, function, chart)

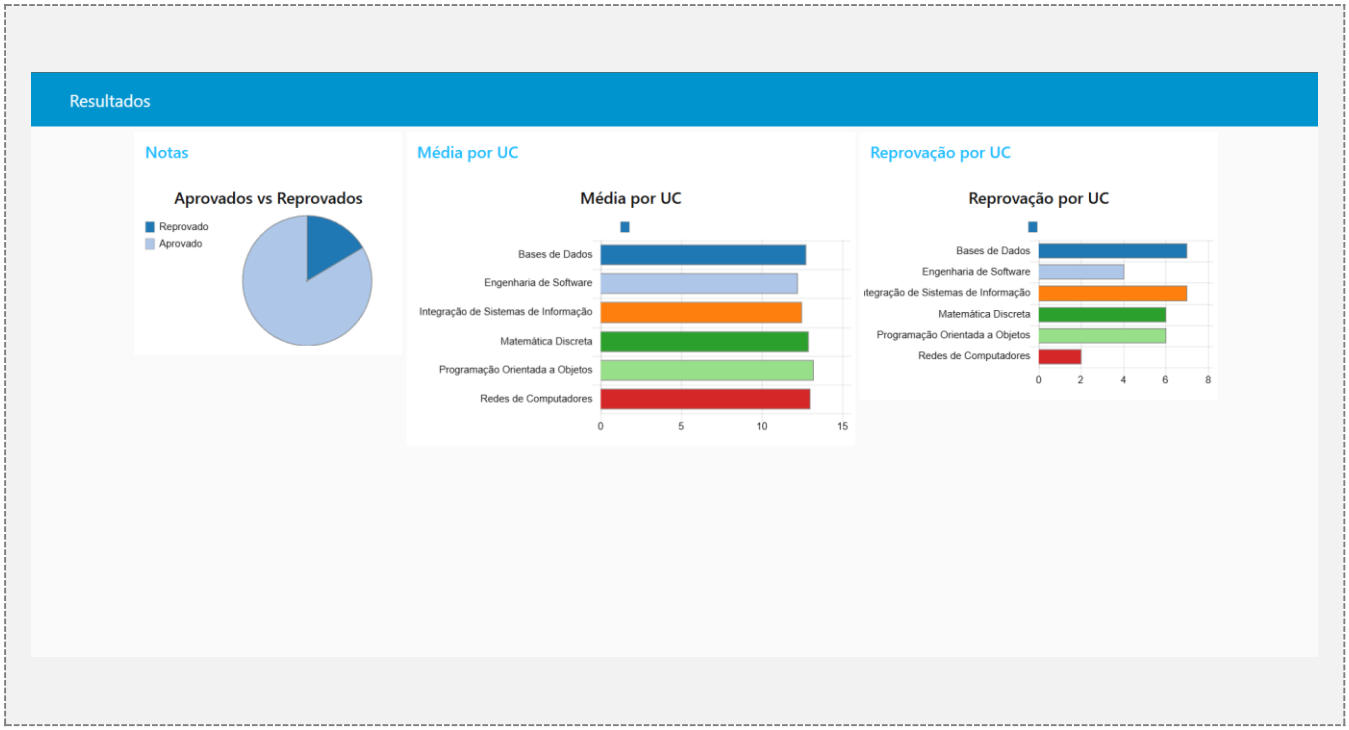


Figura 12 – Dashboard em http://localhost:1880/ui

6. Vídeo com demonstração (QR Code)

O vídeo mostra, por esta ordem: as três fontes de dados; a execução completa do job até ao fim, com todas as entradas a verde; os dados carregados no PostgreSQL (pgAdmin); o email recebido com o anexo; e o dashboard do Node-RED.

Figura 13 – QR Code com a ligação para o vídeo de demonstração

Ligação direta: https://drive.google.com/file/d/1FSuroS-DaTBrt3Jyl7owjVdktuPK33Ls/view?usp=sharing

7. Conclusão e Trabalhos futuros

7.1 Conclusão

O trabalho cumpriu o objetivo de construir um processo ETL completo, desde a recolha em três formatos e várias fontes até à persistência, exportação, notificação e visualização. O processo aplica expressões regulares, validação com ficheiros de erros, junções, agregações e um job com tratamento de falhas. A divisão em três transformações tornou o desenvolvimento mais simples de testar e de corrigir, e permitiu validar cada fase com ficheiros intermédios.

As principais dificuldades e as soluções adotadas foram as seguintes:

Dificuldade	Causa e solução
O REST client não devolvia linhas	O passo exige uma linha de entrada; resolvido com um passo Generate rows
Falhas ao aplicar regex à data	O Kettle inferia o tipo Date; todos os campos do CSV passaram a ser lidos como String
Lookups que só encontravam alguns alunos	Tipos inferidos e diferenças nas chaves; resolvido com campos String, Trim do `username` nos dois lados e desligando a lista ordenada no Stream lookup
Ficheiro de reprovados vazio	A comparação é sensível a maiúsculas (`reprovado` ≠ `Reprovado`); uniformizado o valor por omissão no Number range
Ficheiros que não eram escritos	O Excel bloqueava os CSV abertos; os ficheiros devem estar fechados durante a execução
Erro de SSL no Node-RED	O PostgreSQL local não tem SSL ativo; desativado na ligação do nó `postgresql`
Linhas repartidas entre destinos	Com dois destinos o Kettle distribui as linhas por omissão; usado o modo Copy

7.2 Trabalhos futuros

- Validar também o código da UC das notas contra o ficheiro XML já na transformação `t2`.
- Anonimizar os dados pessoais com técnicas mais fortes, como hashing.

8. Referências bibliográficas

[1] Random User Generator. Documentação da API. https://randomuser.me/documentation (acedido em 22/10/2026).

[2] PostgreSQL Global Development Group. PostgreSQL Documentation. https://www.postgresql.org/docs/ (acedido em 22/10/2026).

[3] PostgreSQL JDBC Driver. https://jdbc.postgresql.org/ (acedido em 22/10/2026).

[4] Node-RED. Documentação. https://nodered.org/docs/ (acedido em 22/10/2026).

[5] node-red-dashboard. https://flows.nodered.org/node/node-red-dashboard (acedido em 22/10/2026).

[6] node-red-contrib-postgresql. https://flows.nodered.org/node/node-red-contrib-postgresql (acedido em 22/10/2026).

[7] Ferreira, L. e Ribeiro, Ó. (2026). Enunciado do Trabalho Prático I, Integração de Sistemas de Informação, IPCA.